

Transformer attention mechanism visualization

👤 I am having trouble visualizing the inputs and outputs of a transformer. I understand that I have a sequence of inputs, they get an embedding, they are multiplied by weights to become keys, values, and queries. But I do not fully understand the outputs and what each hidden state represents. Can you please walk me through an example first conceptually and also visually

11/12/2025, 1:13:47 PM

These were my initial broad questions to Claude to try and help me understand transformers and attention.

I'll walk you through a concrete example both conceptually and visually!

Conceptual Walkthrough

Let's use a simple sentence: **"The cat sat"**

Step 1: Token Embeddings

- "The" → vector of size d (let's say 512 dimensions)
- "cat" → vector of size d (512 dimensions)
- "sat" → vector of size d (512 dimensions)

So you have 3 tokens, each represented as a 512-dimensional vector.

Step 2: Creating Q, K, V Each token's embedding is transformed into three vectors:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What do I contain?"
- **Value (V):** "What information do I actually provide?"

For "cat":

- Q_{cat} asks: "What context is relevant to understanding 'cat'?"
- K_{cat} advertises: "I'm the word 'cat'"

- V_{cat} contains: The actual information about "cat" to be passed forward

Step 3: Attention (The Magic) Each token's query looks at all tokens' keys to decide what's relevant:

- "sat" compares its query against keys of ["The", "cat", "sat"]
- It might find "cat" very relevant (high attention score)
- It might find "The" somewhat relevant (medium score)
- It might find itself "sat" relevant (some score)

Step 4: The Output (Hidden States) Each position produces a **new hidden state** that's a weighted combination of all the values:

For position "sat":

- $Output_{sat} = 0.1 \times V_{The} + 0.7 \times V_{cat} + 0.2 \times V_{sat}$
- This output is a 512-dim vector that now contains information about "sat" **enriched with context from "cat"**

Key Insight: Each output hidden state represents that token **after considering the entire sequence context**. It's no longer just "sat" in isolation—it's "sat (in the context of 'The cat sat')".

Let me create a visual representation:

Transformer Attention Visualization

Interactive Artifact

Here, Claude generated the first iteration of the interactive tool. It only had about 4 slides.

What Each Hidden State Represents

This is the crucial part: **Each output hidden state at position i represents token i after it has "looked at" and gathered information from all other tokens in the sequence.**

Before attention:

- Position 0: just "The"
- Position 1: just "cat"
- Position 2: just "sat"

After attention:

- Position 0: "The" (with context that it precedes "cat sat")
- Position 1: "cat" (with context that it's the subject between "The" and "sat")
- Position 2: "sat" (with strong context about "cat" being the subject)

These contextualized representations can then be:

1. Passed to the next transformer layer (which will further refine them)
2. Used for predictions (like the next token in language modeling)
3. Used for classification (pool all hidden states to make a decision)

The beauty is that this happens in parallel for all positions simultaneously—every token simultaneously gathers context from every other token!

11/12/2025, 1:14:53 PM

👤 This makes sense to me. Follow up questions:

1. What exactly are the trainable parameters?
2. Is there a hidden state for each input vector?
3. How are these hidden states used for prediction or downstream learning?
4. Can you explain autoregressive prediction or masked prediction based on what you have already explained?

Please keep and/or add on to your existing visualization

11/12/2025, 1:17:27 PM Here, after interacting with the first iteration of the tool, I asked follow up questions so that Claude could add more slides. My strategy was not to ask all these questions at the beginning so that Claude could build the tool little by little with more attention to the specific questions I was asking.

Great questions! Let me answer each one and enhance the visualization.

Transformer Attention Visualization • Version 2

Interactive Artifact

11/12/2025, 1:19:49 PM



An error occurred while trying to run the generated artifact. **Unexpected token, expected ";" (4:37)**

Can you fix this error in **transformer_visualization**?

11/12/2025, 1:20:30 PM

There was a syntax error in Claude's code here so I had it regenerate.

Transformer Attention Visualization • Version 3

Interactive Artifact

Fixed! The error was in the last line where the code was cut off. The visualization should now work properly with all 8 steps covering:

1. **Input Tokens** - Starting point
2. **Token Embeddings** - Converting to vectors
3. **Create Q, K, V** - The three transformations
4. **Attention Scores** - How tokens attend to each other
5. **Output Hidden States** - The final contextualized representations (Answer to Q2)
6. **Trainable Parameters** - All the weights that get learned (Answer to Q1)
7. **Using Hidden States** - How to use them for different tasks (Answer to Q3)
8. **Autoregressive vs Masked** - Different attention patterns (Answer to Q4)

Navigate through with the buttons to explore each concept!

11/12/2025, 1:22:15 PM



👤 Going back to the trainable parameters, could you explain what each dimension is and add a slide to show all the math of multiplying everything together and how each vector changes through the transformer. I really want you to focus on dimensions. Can you show this for both single head and multihead attention.

Please keep all the slides you have created so far and add this as additional slides.

11/12/2025, 1:33:07 PM

Here, I really wanted to dig into the matrix dimensions in the transformer and see how the vectors change over time. I also wanted to compare the vector dimensions across single and multiheaded attention.

Transformer Attention Visualization • Version 4

Interactive Artifact

Transformer Attention Visualization • Version 5

Interactive Artifact

Transformer Attention Visualization • Version 6

Interactive Artifact

Perfect! I've added three new detailed slides that focus on dimensions:

New Slides Added:

Step 7: Single-Head Attention Math

- Shows exact dimensions at each step
- Traces how $[3, 512]$ becomes $[3, 64]$ through Q, K, V projections
- Shows the $[3, 3]$ attention matrix computation
- Final output $[3, 64]$

Step 8: Multi-Head Attention Math

- Explains the parallel nature of 8 heads
- Shows how each head operates independently on 64-dim subspace
- Demonstrates concatenation: $8 \times [3, 64] \rightarrow [3, 512]$
- Final W_O projection: $[3, 512] @ [512, 512] \rightarrow [3, 512]$

Step 9: Complete Dimension Flow Through Transformer

- Traces dimensions through an entire transformer block
- Shows Multi-head attention dimensions
- Shows Feed-forward network expansion and contraction: $512 \rightarrow 2048 \rightarrow 512$
- Highlights residual connections that maintain $[3, 512]$ throughout
- Complete flow summary at the end

All your previous slides are preserved! Navigate through to see the new dimension-focused content. The visualization now covers everything from basic concepts to the detailed mathematical dimensions of both single-head and multi-head attention.

11/12/2025, 1:34:35 PM

 Follow up:

1. When we do $\text{output} = \text{attention_weights} @ V$, we are doing a linear combination of the different values based on the attention weights to get the hidden state. This means the softmax is across the rows?
2. How does each head learn to attend to a different part of the input? I've seen an example where the input embeddings are divided into different sections depending on the number of heads (taking the first $\text{embedding_dim}/\text{heads}$ for each separate head. it seems you are projecting them all down, so how do you make sure that each projection is different?
3. For single head math, why did we only end up with an output dimension of 64 in the example but with multihead we are back at 512?

11/12/2025, 1:48:00 PM

Here, I was a bit confused why the single head attention output wasn't scaled up to the input dimension while the multihead attention output was. I also wanted to make sure I understood how the value vectors were combined. Also, I asked how each of the weight matrices for each of the heads in multiheaded attention learned different aspects of the data.

Great questions! Let me update the visualization to address these important points.

Transformer Attention Visualization • Version 7

Interactive Artifact

Transformer Attention Visualization • Version 8

Interactive Artifact

Transformer Attention Visualization • Version 9

Interactive Artifact

Perfect! I've added three new slides that directly answer your questions:

New Slides:

Step 10: Understanding Softmax & Attention Direction (Answers Q1)

- Shows the attention matrix with actual numbers
- Demonstrates that softmax IS applied across rows (each row sums to 1)
- Shows how each row creates a weighted combination of value vectors
- Visual breakdown of matrix multiplication: $\text{attention_weights} @ V$

Step 11: How Each Head Learns Different Patterns (Answers Q2)

- Clarifies that heads do NOT divide the embedding (common misconception!)
- Shows that each head has its own separate W_Q , W_K , W_V matrices
- All heads see the full 512-dim input but project it differently
- Explains how specialization emerges through random initialization and gradient descent
- Examples of what different heads might learn (syntax, semantics, long-range deps)

Step 12: Single-Head vs Multi-Head Output Dimensions (Answers Q3)

- Shows why single-head outputs only 64 dimensions
- Shows how multi-head concatenation gives us 512: $8 \times 64 = 512$
- Explains the key formula: $d_k = d_{\text{model}} / \text{num_heads}$
- Side-by-side comparison showing the dimensional flow
- Clarifies this is an intentional design choice

All your original slides are still there! The visualization now has 14 comprehensive steps covering everything from basics to these nuanced details about attention mechanics.

11/12/2025, 1:49:59 PM

Because Claude had done so much and there was a lot of math, I wanted to make sure it didn't make a mistake in its explanation. To this end, I copied the entire source code for the visualization into GPT-5 and got the following feedback. For the most part, GPT found the visualization good but it pointed out a couple of assumptions Claude made that could confuse the user and suggested corrections. I took the GPT suggestions and prompted Claude to fix the visualization.

👤 I ran your code through another LLM to check for correctness. Here was the feedback. Please tell me if you think this needs tweaking and if so please add the updates:

Short version: almost all of the shapes and operations are right. A few fixes/clarifications:

- Output width equals d_v , not d_k . In Step 12 you say single-head “outputs 64 because that’s its d_k .” The output dimension after `attn @ V` is the width of V, i.e., d_v . You set `d_k = d_v = 64` earlier, so numerically it’s fine, but the reason should be “because $d_v=64$.”
- Single-head normally still has an output projection. You note “we’d need to project back to d_{model} .” Correct—and in standard implementations there is a `W_0 ∈ ℝ^{d_v × d_model}` even for a single head, so the block returns `[seq, d_model]`.
- Include biases (and often GELU). Your parameter list omits biases for `W_Q/W_K/W_V/W_0` (many libs include them) and the FFN activation is typically GELU (ReLU is acceptable but not the usual default). You do include `b1/b2` later; consider listing them with the trainable parameters section for completeness.
- Scaling term mention is inconsistent. You correctly use `scores = Q KT / √d_k` in Step 7. In Step 10 you write `scores = Q @ K.T` without the scale. Add `/ sqrt(d_k)` there for consistency.
- “Each head sees the full X” → clarify wording. Conceptually true: each head projects the full `[seq, d_model]` with its own `W_Q/W_K/W_V`. Your “NOT dividing embeddings” box is right, but some efficient implementations do a single big projection and then chunk per head; that’s still equivalent to per-head projections, not a raw channel split of X.
- Residual/LayerNorm ordering. You show post-LN (`Add → LayerNorm`). Many modern models use pre-LN (`LayerNorm → sublayer → Add`). Your math is consistent internally, just note it’s one of the two common variants.
- Softmax orientation is correct. “Across rows” (per-query distribution) is exactly right, and your example matrices sum to 1 by row.

- Autoregressive mask example is fine. Your toy causal matrix respects  (self-attention allowed). Good. Optional polish (not errors):
- Position embeddings can be learned or sinusoidal; you already hint at this.
- Mention the LM head / weight tying (projection to vocab, often tied with token embeddings). If you tweak the `d_v` phrasing, add the scale in Step 10, and note the WO/biases, the math checks out.

11/12/2025, 2:06:21 PM

Great feedback! Let me address these points. Most of it is correct, but there are a few clarifications and additions needed. Let me update the visualization:

Transformer Attention Visualization • Version 10

Interactive Artifact

Transformer Attention Visualization • Version 11

Interactive Artifact

Transformer Attention Visualization • Version 12

Interactive Artifact

Transformer Attention Visualization • Version 13

Interactive Artifact

Transformer Attention Visualization • Version 14

Interactive Artifact

Transformer Attention Visualization • Version 15

Interactive Artifact

Transformer Attention Visualization • Version 16

Interactive Artifact

Transformer Attention Visualization • Version 17

Interactive Artifact

Transformer Attention Visualization • Version 18

Interactive Artifact

Transformer Attention Visualization • Version 19

Interactive Artifact

Transformer Attention Visualization • Version 20

Interactive Artifact

Transformer Attention Visualization • Version 21

Interactive Artifact

Transformer Attention Visualization • Version 22

Interactive Artifact

Transformer Attention Visualization • Version 23

Interactive Artifact

Transformer Attention Visualization • Version 24

Interactive Artifact

Transformer Attention Visualization • Version 25

Interactive Artifact

Perfect! I've updated the visualization to address all the feedback points:

Updates Made:

1. **Output dimension clarification (d_v not d_k):** Fixed Step 7, 11, and 12 to correctly state that output dimension after attn @ V is d_v (the width of V), not d_k .
2. **Single-head W_O projection:** Added clarification that single-head attention DOES use W_O : $[3, 64] @ [64, 512] = [3, 512]$ to project back to d_{model} .
3. **Biases added:** Updated the trainable parameters section to include biases for W_Q , W_K , W_V , W_O , and the FFN biases b_1 , b_2 .
4. **GELU activation:** Changed FFN description from generic "activation" to specifically mention GELU as the typical choice (noting ReLU is also used).
5. **Scaling term consistency:** Added the $/\sqrt{d_k}$ scaling in Step 10's attention score computation to be consistent with Step 7.
6. **"Each head sees full X" clarification:** Added a new section explaining that conceptually each head sees the full input, and noted that some efficient implementations do batched projections but this is mathematically equivalent.
7. **LayerNorm ordering note:** Added a note about Post-LN vs Pre-LN variants, explaining that the visualization shows Post-LN but Pre-LN is also common in modern models.
8. **Softmax direction:** Already correct, but made it even more explicit with $\text{dim}=-1$ notation.

The math is now fully accurate and the explanations are more precise! All the original content is preserved, just enhanced with these corrections.

11/12/2025, 2:09:17 PM